Observe in the previous example that consecutive separators result in empty arrays.

Defined in `active_support/core_ext/array/grouping.rb`.

11 Extensions to `Hash`

11.1 Conversions

11.1.1 `to_xml`

The method `to_xml` returns a string containing an XML representation of its receiver:

```
{"foo" => 1, "bar" => 2}.to_xml
# =>
# <?xml version="1.0" encoding="UTF-8"?>
# <hash>
#   <foo type="integer">1</foo>
#   <bar type="integer">2</bar>
# </hash>
```

To do so, the method loops over the pairs and builds nodes that depend on the *values*. Given a pair `key`, `value`:

- If `value` is a hash there's a recursive call with `key` as `:root`.
- If `value` is an array there's a recursive call with `key` as `:root`, and `key` singularized as `:children`.
- If `value` is a callable object it must expect one or two arguments. Depending on the arity, the callable is invoked with the `options` hash as first argument with `key` as `:root`, and `key`singularized as second argument. Its return value becomes a new node.
- If `value` responds to `to_xml` the method is invoked with `key` as `:root`.
- Otherwise, a node with `key` as tag is created with a string representation of `value` as text node. If `value` is `nil` an attribute "nil" set to "true" is added. Unless the option `:skip_types`exists and is true, an attribute "type" is added as well according to the following mapping:

```
XML_TYPE_NAMES = {
  "Symbol"   => "symbol",
  "Fixnum"   => "integer",
  "Bignum"   => "integer",
  "BigDecimal" => "decimal",
  "Float"    => "float",
  "TrueClass" => "boolean",
  "FalseClass" => "boolean",
  "Date"     => "date",
  "DateTime"  => "datetime",
  "Time"     => "datetime"
}
```

By default the root node is "hash", but that's configurable via the `:root` option.

The default XML builder is a fresh instance of `Builder::XmlMarkup`. You can configure your own builder with the `:builder` option. The method also accepts options like `:dasherize` and friends, they are forwarded to the builder.

Defined in `active_support/core_ext/hash/conversions.rb`.

11.2 Merging

Ruby has a built-in method `Hash#merge` that merges two hashes:

```
{a: 1, b: 1}.merge(a: 0, c: 2)
# => {:a=>0, :b=>1, :c=>2}
```

Active Support defines a few more ways of merging hashes that may be convenient.